

가중치를 줄이고 실행 결과를 확인하다

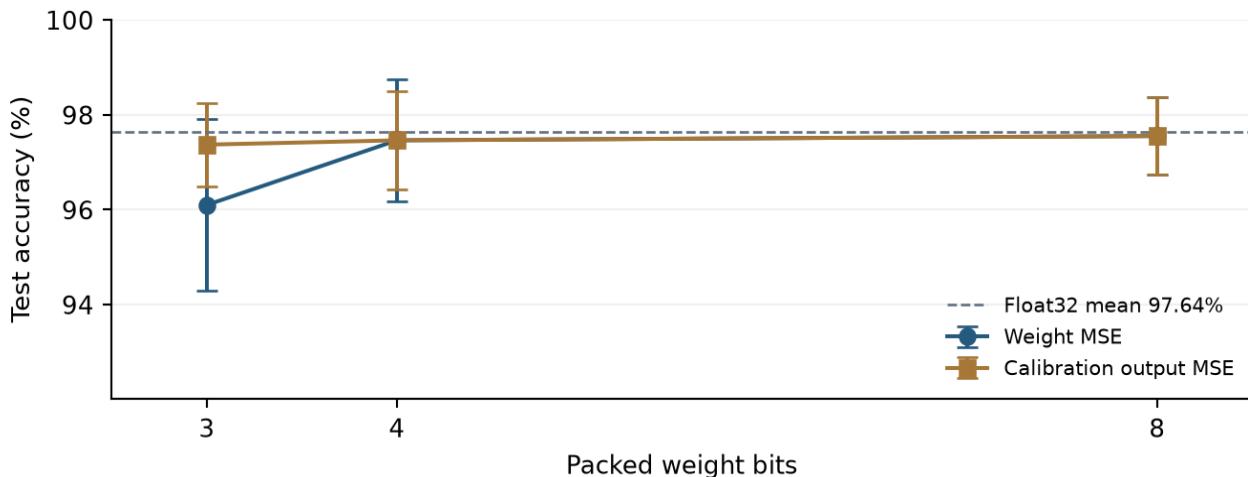
TinyQuant Lab · EvalReceipt

배경한 / Kyunghan Bae

숫자 이미지 분류 모델을 직접 학습해 압축 조건을 비교했습니다. 평가 실행 기록과 파일을 확인하는 도구를 만들고, 저장한 모델 21개를 다시 실행해 실제 모델 평가에 적용했습니다.

모델 압축 실험

NumPy로 64→48→10 분류 모델을 학습했습니다. digits 데이터 1,797개를 시드 17·43·91마다 학습 1164개, 보정 266개, 시험 367개로 나눴습니다. 픽셀을 16으로 나누고 350에포크(학습 데이터 350번 반복)를 학습했습니다. 시험 데이터는 마지막 정확도 계산에 사용했습니다.



오차 막대는 세 시드의 표본 표준편차입니다. 세로축은 관측 정확도 부근을 확대했습니다. 시험 세트는 시드마다 다시 분리되므로 오차 막대는 서로 다른 학습·분리 결과의 변동을 함께 담습니다.

4비트 보정 출력 방식의 평균 시험 정확도는 **97.46%**였습니다. float32 평균 **97.64%**와의 차이는 **-0.18퍼센트포인트**였습니다. 패킹한 가중치 저장 크기는 8.00배, 스케일·편향을 포함한 배열 크기는 6.45배 줄었습니다.

실행 기록 검사

EvalReceipt는 정상 실행된 오답과 실행 결함을 구분합니다. 별도 프로세스와 파일 기록을 사용한 20개 시나리오에서 예상 결과를 확인했습니다. 정상 제어 3개를 유지하고, 주입한 결함 17개를 검출했습니다. 이후 모델 21개의 실제 재실행 결과에 적용했습니다.

압축 조건은 무엇으로 고를까

같은 후보 격자, 두 목적 함수, 분리한 시험 데이터

가중치 오차 방식은 각 층에서 원래 가중치와 복원 가중치의 평균 제곱 오차(MSE)를 최소화합니다. 보정 출력 방식은 두 층을 함께 선택하며, 원래 모델과 압축 모델이 보정 이미지에 내는 마지막 출력값의 MSE를 최소화합니다. 0.45부터 1.00까지 12개 클리핑 비율을 사용해 각각 층별 12개, 두 층 조합 144개를 비교했습니다.

행렬 대칭 양자화를 사용했습니다. 가중치는 8·4·3비트 정수 코드로 패킹하고 스케일과 편향은 float32로 저장했습니다. 추론 때 정수 가중치를 풀어 float32 행렬 연산에 넣었습니다.

방식 / 비트	정확도 평균 ± SD	가중치 B	배열 B	파일 B	추론 ms
float32 / 32	97.64 ± 0.83%	14,208	14,440	15,693	113.730
가중치 / 8	97.55 ± 0.82%	3,552	4,016	5,881	53.754
보정 출력 / 8	97.55 ± 0.82%	3,552	4,016	5,881	59.232
가중치 / 4	97.46 ± 1.29%	1,776	2,240	4,107	52.011
보정 출력 / 4	97.46 ± 1.03%	1,776	2,240	4,106	48.816
가중치 / 3	96.09 ± 1.81%	1,332	1,796	3,667	49.983
보정 출력 / 3	97.37 ± 0.88%	1,332	1,796	3,666	45.661

정확도는 세 시드의 평균과 표본 표준편차입니다. 파일 크기와 추론 시간은 세 시드 평균입니다. 추론 시간은 367개 시험 이미지 배치의 시드별 중앙값을 평균했습니다. 파일 읽기를 제외하고 압축 해제·복원·행렬 계산을 포함했습니다.

출력 오차와 정확도를 따로 확인한 이유

4비트에서 보정 출력 MSE는 가중치 기준 0.5962, 출력 기준 0.2843였습니다. 시험 정확도 차이(출력 기준 - 가중치 기준)는 +0.00퍼센트포인트였습니다. 출력값 오차와 최종 분류 결과를 함께 비교했습니다.

3비트에서 보정 출력 MSE는 가중치 기준 2.5601, 출력 기준 1.4800였습니다. 시험 정확도 차이(출력 기준 - 가중치 기준)는 +1.27퍼센트포인트였습니다. 출력값 오차와 최종 분류 결과를 함께 비교했습니다.

이 결과는 1,797개 숫자 이미지와 작은 분류 모델에 대한 실험입니다. 세 시드의 시험 집합은 서로 겹칠 수 있습니다. 같은 구조의 모델을 시드 별로 학습했고, 최종 시험 결과로 압축 조건을 다시 고르지 않았습니다.

좋은 점수보다 먼저 확인한 것

프로세스, 예정 사례, 출력 파일의 실제 일치

검사 입력은 예정 테스트 ID, 실행 시각, 종료 코드, 시간 초과 여부, 점수와 파일 기록입니다. 전체 예정 테스트를 성공률의 분모로 유지하고, 정상 실행된 응답은 유효한 실행의 점수 0으로 기록했습니다. 실패·누락 사례는 정상 완료에 넣지 않았습니다.

실험	실제 검사 결과	검출 항목
정상 정답	정상 기록	정상 실행
정상 오답	정상 기록	정상 실행
출력 전 중단	결함 검출	incomplete_execution, missing_artifact, missing_passed, missing_score, process_failure
출력 후 중단	결함 검출	incomplete_execution, process_failure
시간 초과	결함 검출	incomplete_execution, missing_artifact, missing_passed, missing_score, process_failure, timeout
사례 누락	결함 검출	missing_case
사례 중복	결함 검출	duplicate_case
파일 변조	결함 검출	hash_mismatch, stale_artifact_mtime
이전 파일 재사용	결함 검출	stale_artifact_mtime, stale_artifact_timestamp
KLayout 면적 계산	정상 기록	정상 실행
KLayout 입력 오류	결함 검출	incomplete_execution, missing_artifact, missing_passed, missing_score, process_failure

전체 20개에는 잘못된 JSON, 유효하지 않은 점수, 빈 예정 목록, 추가 사례, 점수·통과 판정 불일치, 파일 삭제와 잘못된 경로도 포함됩니다. 시나리오별 JSON과 실제 프로세스 출력을 원본 결과 폴더에 보관했습니다.

실제 기하 계산과 오류

KLayout 0.30.5의 Region으로 두 100×100 사각형을 50만큼 어긋나게 배치해 교집합 면적 5,000을 계산했습니다. 별도 사례에서는 Region에 잘못된 입력을 넘겨 Python 바인딩의 TypeError와 비정상 종료 코드를 확인했습니다.

원본 보존을 확인한 수정

독립 검사에서 결과 보고서 경로가 잘못 선언된 입력 파일과 같을 때 원본을 덮어쓸 수 있는 결함을 발견했습니다. 선언 경로와 실제 대상 경로를 모두 보호하도록 수정하고, 절대 경로·상위 경로·심볼릭 링크 별칭에 대한 회귀 검사를 추가했습니다. 수정한 설치 파일을 별도 폴더에 설치해 같은 검사를 통과했습니다.

두 프로젝트를 함께 실행하다

저장 모델 21개 재검사와 재실행 자료

TinyQuant Lab의 저장 모델 21개를 별도 프로세스로 다시 추론했습니다. 실제 시험 정확도와 모델 해시를 새 실행 기록에 넣고 EvalReceipt로 검사했습니다. 원본 모델은 유지하고 실험 폴더의 복사본을 사용했습니다.

적용 실험	기록 완전성	예상 결과 확인
clean	정상	통과
tampered-model	결함 검출	통과
missing-record	결함 검출	통과

검증 자료

EvalReceipt의 단위·CLI 테스트 40개와 별도 원본·설치 파일 검사 12개를 통과했습니다. TinyQuant Lab의 비트 디코딩, 미분 계산, 저장·복원 검사를 별도로 수행했습니다. 저장 모델은 구현 모듈을 가져오지 않는 독립 디코더와 행렬 계산으로 다시 확인했습니다. 재계산 자료는 소스 패키지와 함께 보관했습니다.

재실행 명령

전체 소스 압축을 풀고 research-projects 폴더에서 시작합니다. 새 학습 결과는 reproduced-results에 저장해 기존 측정 기록을 유지합니다.

```
cd quantization-lab
python -m pip install -r requirements.txt
python tinyquant.py experiment --output reproduced-results --seeds 17 43 91
python rebenchmark.py reproduced-results

cd ../eval-receipt
python scripts/make_fixtures.py
python -m evalreceipt fixtures/clean.json --json-out audit.json --html-out audit.html
```

TinyQuant Lab 소스에는 공식 데이터의 저장본과 해시, 모델·분리 인덱스·선택 기록이 들어 있습니다. EvalReceipt 소스에는 입력 형식, 예제, 검사 코드와 실제 프로세스 실험을 넣었습니다. 학습은 고정 350회이며 모델 저장과 실행 시간 측정을 구분했습니다.

자료와 참고 연구

Kim et al., GuidedQuant, ICML 2025. 최종 손실을 고려하는 양자화 문제에서 착안해 작은 모델의 독립 실험을 설계했습니다.
arxiv.org/abs/2505.07004

scikit-learn digits 데이터. 공식 저장소의 digits.csv.gz에서 1,797개 레코드를 읽었습니다.
`scikit-learn / load_digits`

Rule2DRC 평가 문제를 바탕으로 실행 실패와 빈 예측을 검사하는 별도 도구를 만들었습니다.
github.com/snu-mllab/Rule2DRC